Respostas dos Desafios — Blockchain

Prática 1 — Blockchain do Zero em Python

Desafio 1 — Por que is_valid() retorna False mesmo após uma única alteração?

A blockchain é validada comparando o hash armazenado de cada bloco com o hash que seria calculado novamente a partir dos seus dados. Quando o campo data de um bloco intermediário é alterado, o conteúdo usado para calcular o hash também muda. Consequentemente, o novo hash calculado fica diferente do hash que estava armazenado no bloco.

> Além disso, o bloco seguinte guarda em prev_hash o hash antigo do bloco anterior. Portanto, a alteração quebra a integridade do próprio bloco adulterado e também o elo com o bloco seguinte. É por isso que uma única modificação já faz a validação da cadeia falhar.

```
Antes da adulteração: False
Depois da adulteração: False
Hash antigo do bloco 1: c6e25e9631839756 ...
```

A ideia demonstrada é a imutabilidade: alterar um bloco modifica sua impressão digital criptográfica e rompe a sequência dos blocos seguintes. O notebook define exatamente essa relação entre SHA-256, imutabilidade e encadeamento.

Prática 2 — Prova de Trabalho (Proof of Work)

Desafio 2 — O que acontece com dificuldade 6?

Quanto maior a dificuldade, maior é a quantidade esperada de tentativas necessárias para encontrar um nonce cujo hash comece com a quantidade exigida de zeros. No teste do notebook, a dificuldade 3 encontrou um resultado rapidamente, enquanto a dificuldade 4 exigiu mais tentativas e a dificuldade 5 exigiu milhões de tentativas.

```
Dificuldade 3: nonce=   596  tempo=0.00s
Dificuldade 4: nonce=146909  tempo=0.70s
Dificuldade 5: nonce=2865608  tempo=14.49s
```

> Com dificuldade 6, o programa precisa procurar um hash começando com seis zeros. Em média, a chance de cada tentativa é de 1 em 16^6, ou seja, aproximadamente 1.048.576 tentativas. Como cada tentativa recalcula o SHA-256, o tempo de mineração aumenta bastante em relação à dificuldade 5. O valor exato do nonce e o tempo não são fixos, pois dependem da sequência de tentativas.

Portanto, a principal conclusão do desafio é: aumentar a dificuldade aumenta o trabalho computacional necessário para minerar o bloco. Isso torna mais caro refazer uma sequência de blocos em um sistema baseado em Proof of Work.

Prática 3 — Assinaturas Digitais e Carteiras

Desafio 3 — O que acontece se Mallory usar a chave pública errada?

A assinatura de Alice foi criada usando a chave privada de Alice. Para verificar essa assinatura corretamente, deve-se utilizar a chave pública correspondente à chave privada de Alice. Mallory possui outro par de chaves, portanto sua chave pública não corresponde à assinatura produzida por Alice.

```
Mallory consegue verificar? False
Verificação de tx adulterada: False
```

Assim, quando Mallory tenta verificar a assinatura usando a própria chave pública, o resultado é False. Isso demonstra que possuir uma chave pública diferente não permite falsificar ou validar uma assinatura produzida por outra chave privada.

> O segundo teste altera a transação de 100 para 999. A assinatura foi criada sobre o conteúdo original. Como a mensagem mudou, seu hash também muda e a assinatura deixa de corresponder à mensagem. Por isso, a verificação da transação adulterada também retorna False.

Em resumo, o desafio demonstra duas propriedades importantes: autenticidade, porque a assinatura está vinculada à chave privada de Alice; e integridade, porque modificar a mensagem invalida a assinatura.

Prática 4 — Smart Contracts (simulado)

Desafio 4 — Bob deve conseguir transferir mais tokens do que possui?

Não. O contrato deve recusar a transferência. Antes de alterar os saldos, o método transfer() verifica se o saldo do remetente é suficiente:

```
if self.balances.get(sender, 0) < amount:
    return False
```

Bob recebeu 250 tokens. O desafio tenta fazer Bob transferir 999.999 tokens para Alice. Como 250 < 999.999, a condição é verdadeira e a função retorna False imediatamente. Nenhum saldo é alterado.

```
Tentativa de transferência fraude OK? False
Saldo de Bob (deve continuar igual): 250
```

> O comportamento é equivalente, na simulação, ao require usado no Solidity: a operação só pode continuar se o remetente possuir saldo suficiente. Dessa forma, a tentativa de transferir uma quantidade maior que o saldo é rejeitada.

Prática 5 — Web3: Conectando à Blockchain Real

Desafio 5 — Consultar saldo, bloco mais recente e total supply do USDC

1. Consultar o próprio saldo: o notebook fornece a função consultar_saldo(), que usa w3.eth.get_balance(endereco) quando existe uma conexão real com a Ethereum. É necessário fornecer um endereço de carteira e uma URL RPC configurada.

> Na execução realizada, a URL ALCHEMY_URL estava vazia, portanto o notebook utilizou o modo demo. O valor 1.2345 ETH apresentado é fictício e não representa o saldo real da carteira.

```
endereco: 0xd8dA6BF26964aF9D7eEd9e03E53415D37aA96045
eth: 1.2345
modo: demo (configure ALCHEMY_URL para dados reais)
```

2. Ler o bloco mais recente: com uma conexão real, a chamada indicada pelo notebook é:

```
latest = w3.eth.get_block('latest')
print(latest.number, latest.hash.hex())
```

Essa chamada permite obter o número, o hash e outras informações do bloco mais recente da rede. Como não havia conexão RPC configurada durante a execução, o notebook exibiu os dados fictícios do modo demo:

```
[demo] Bloco #19000000 (fictício)
[demo] Hash: 0xabc123...
```

3. Consultar o total supply do USDC: o desafio informa o endereço do contrato USDC e pede uma chamada ao contrato para obter o totalSupply(). Para fazer essa consulta de forma real, seria necessário estar conectado à mainnet e instanciar o contrato usando uma ABI que contenha essa função.

Portanto, com o notebook nas condições executadas, não foi possível obter o total supply real do USDC. O material fornece o endereço e orienta a implementação, mas a execução estava em modo demo e não apresenta um resultado real para essa consulta.

Resumo das respostas

Desafio	Resposta
1	Uma alteração muda o hash do bloco e quebra o elo com os blocos seguintes; por isso a cadeia deixa de ser váli
2	A dificuldade 6 exige, em média, cerca de 1.048.576 tentativas e tende a demorar mais que a dificuldade 5.
3	A chave pública de Mallory não valida a assinatura de Alice; alterar a mensagem também invalida a assinatura.
4	Bob não consegue transferir 999.999 tokens possuindo apenas 250; a operação retorna False e o saldo permane
5	Saldo e bloco real exigem RPC; o notebook executou em modo demo. O total supply do USDC não foi obtido na e